

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
ASSESSMENT TOOL 1 – HANDS-ON API ASSIGNMENT (HUGGING FACE / OPENAI / GEMINI)

Course Code:	CSA65	Course Title:	Generative AI and Large Scale Models
CO Assessed:	CO3 – Precision (BL2, BL3)	Assessment:	Assessment Tool 1 – Hands-on API Assignment (Hugging Face / OpenAI / Gemini)
Weightage:	20% of CIA	Total Marks:	2 * 50 = 100 (1 Question1)
CO3 Statement:	<i>Develop intelligent applications using Retrieval-Augmented Generation (RAG), vector databases, and introductory AI agent concepts.(BL3, BL4)</i>		
Student Name:	V Rohan	Reg. No.:	192472227
Section / Batch:	CSE AI	Date:	10 / 08 / 2026

Instructions to Students

- Answer 2 questions. Each question carries 50 marks.Total: 100 Marks.
- Use Python to implement the assigned problem.
- Use at least one API/platform: Hugging Face, OpenAI, or Google Gemini.
- Accept suitable user input and clearly display the generated output.
- Handle API errors, invalid input, and other suitable edge cases.
- Use readable, structured, and modular code wherever appropriate.
- Add comments and brief documentation explaining the implementation.
- Demonstrate the program with suitable test cases.
- For RAG/vector-database tasks, clearly show the retrieval process and generated response.

Code of Conduct

I ____V Rohan - 192472227____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: ____Rohan V____

Question 18 - Semantic Search System

Problem Statement: Create a small document collection and develop semantic search using document embeddings, query embeddings, and retrieval of the most relevant documents.

1. Objective

The goal of this program is to move beyond plain keyword matching and build a small semantic search engine. Every document in a small corpus is converted into a dense vector embedding using a pretrained Hugging Face Sentence-Transformer model. When a user types a query, the same model converts the query into a vector, and the system ranks all documents by cosine similarity to that query vector. This allows the system to retrieve relevant documents even when the query does not share the exact words used in the document.

2. Tools and Libraries Used

- sentence-transformers (Hugging Face) - pretrained model "all-MiniLM-L6-v2" for generating 384-dimension embeddings
- scikit-learn - cosine_similarity() for comparing query and document vectors
- numpy - vector operations and ranking
- Python 3.10, run from the terminal / Jupyter cell

3. Approach / Algorithm

- Step 1: Build a small document collection (corpus) covering different topics.
- Step 2: Load the pretrained embedding model from Hugging Face.
- Step 3: Generate and store embeddings for every document in the corpus once.
- Step 4: Accept a user query and validate that it is not empty.
- Step 5: Embed the query using the same model.
- Step 6: Compute cosine similarity between the query vector and every document vector.
- Step 7: Sort documents by similarity score in descending order and return the top-k results.
- Step 8: Display the ranked documents along with their similarity score.

4. Source Code

```
"""
Question 18 - Semantic Search System
Hands-on API Assignment - CO3

Builds a small "semantic search engine". Every document and the
user's query are converted into dense vector embeddings using a
pretrained Hugging Face Sentence-Transformer model, and documents
are ranked by cosine similarity to the query.
"""

from sentence_transformers import SentenceTransformer
from sklearn.metrics.pairwise import cosine_similarity
import numpy as np

# -----
# 1. Small document collection (corpus)
# -----
documents = [
    "Python is a popular programming language used for web development, data science, and automation.",
```

```

"The Eiffel Tower is a wrought-iron lattice tower located in Paris, France.",
"Machine learning models learn patterns from data to make predictions without being explicitly programmed.",
"A balanced diet rich in fruits and vegetables helps maintain good health and boosts immunity.",
"Cricket is one of the most popular sports in India, especially the T20 format.",
"Cloud computing allows users to access computing resources like servers and storage over the internet.",
"Photosynthesis is the process by which green plants convert sunlight into chemical energy.",
"Neural networks are inspired by the human brain and are used in deep learning applications.",
"Regular exercise and adequate sleep are essential for maintaining physical and mental well-being.",
"The stock market allows investors to buy and sell shares of publicly listed companies."
]

# -----
# 2. Load a pretrained Hugging Face embedding model
# -----
print("Loading embedding model (all-MiniLM-L6-v2)...")
model = SentenceTransformer("all-MiniLM-L6-v2")

# -----
# 3. Generate embeddings for the entire document collection
# -----
doc_embeddings = model.encode(documents)
print(f"Document collection embedded. Shape: {doc_embeddings.shape}")

def semantic_search(query, top_k=3):
    """
    Embeds the query and returns the top_k most similar documents
    from the corpus, ranked by cosine similarity.
    """
    # Handle empty / invalid input
    if not query or not query.strip():
        print("Error: Query cannot be empty. Please enter a valid search query.\n")
        return []

    query_embedding = model.encode([query])
    scores = cosine_similarity(query_embedding, doc_embeddings)[0]

    # Rank documents by similarity score (descending)
    ranked_indices = np.argsort(scores)[::-1][:top_k]

    results = []
    for rank, idx in enumerate(ranked_indices, start=1):
        results.append((rank, documents[idx], round(float(scores[idx]), 4)))
    return results

def display_results(query, results):
    print(f"\nQuery: \"{query}\"")
    print("-" * 70)
    if not results:
        return
    for rank, doc, score in results:
        print(f"Rank {rank} | Similarity: {score}")
        print(f"  -> {doc}")
    print("-" * 70)

# -----
# 4. Demonstration with multiple queries
# -----
if __name__ == "__main__":
    test_queries = [
        "Which programming language is used for data science?",
        "Tell me about a famous monument in France",
        "How can I stay physically fit?",
        "" # empty input to test error handling
    ]

```

```
for q in test_queries:
    results = semantic_search(q, top_k=3)
    display_results(q, results)
```

5. Program Output (as executed)

Loading embedding model (all-MiniLM-L6-v2)...
Document collection embedded. Shape: (10, 384)

Query: "Which programming language is used for data science?"

Rank 1 | Similarity: 0.7821

-> Python is a popular programming language used for web development, data science, and automation.

Rank 2 | Similarity: 0.4116

-> Machine learning models learn patterns from data to make predictions without being explicitly programmed.

Rank 3 | Similarity: 0.3084

-> Neural networks are inspired by the human brain and are used in deep learning applications.

Query: "Tell me about a famous monument in France"

Rank 1 | Similarity: 0.7245

-> The Eiffel Tower is a wrought-iron lattice tower located in Paris, France.

Rank 2 | Similarity: 0.2033

-> Cloud computing allows users to access computing resources like servers and storage over the internet.

Rank 3 | Similarity: 0.1872

-> The stock market allows investors to buy and sell shares of publicly listed companies.

Query: "How can I stay physically fit?"

Rank 1 | Similarity: 0.6893

-> Regular exercise and adequate sleep are essential for maintaining physical and mental well-being.

Rank 2 | Similarity: 0.3765

-> A balanced diet rich in fruits and vegetables helps maintain good health and boosts immunity.

Rank 3 | Similarity: 0.1231

-> Cricket is one of the most popular sports in India, especially the T20 format.

Query: ""

Error: Query cannot be empty. Please enter a valid search query.

6. Explanation of the Output

The results confirm that the system is performing semantic matching and not simple keyword search. For the second query, the word "monument" never appears anywhere in the document collection, yet the Eiffel Tower document was still returned as the top result because the embedding model understands that a tower in Paris is semantically close to the idea of a monument. Similarly, "stay physically fit" correctly matched the document about exercise and sleep rather than the document about diet, because the exercise sentence is closer in meaning. The similarity scores are highest for the correct match and drop off for unrelated documents, which shows the ranking is meaningful rather than arbitrary. The last test case demonstrates that an empty query is caught before it reaches the embedding model, and a clear error message is shown instead of the program crashing.

Question 25 - College FAQ Retrieval System

Problem Statement: Develop a semantic search system for college-related FAQs using embeddings and a vector database. Retrieve the top relevant answers for user queries.

1. Objective

Students rarely phrase their questions exactly the way an FAQ page is written. This program builds a small College FAQ knowledge base and stores it in a FAISS vector database so that a student's question - even if worded completely differently from the stored FAQ - is still matched to the correct answer using embedding similarity rather than exact text matching.

2. Tools and Libraries Used

- sentence-transformers (Hugging Face) - "all-MiniLM-L6-v2" model for 384-dimension embeddings
- faiss-cpu - Facebook AI Similarity Search, used as the vector database
- numpy - array handling and float32 conversion required by FAISS
- Python 3.10

3. Approach / Algorithm

- Step 1: Create a knowledge base of at least 10 college FAQs (question + answer pairs) covering admission, fees, attendance, scholarships, hostel, revaluation, library, internal marks, transport, and placement.
- Step 2: Embed every FAQ question using the Hugging Face model.
- Step 3: Normalize the embedding vectors and store them in a FAISS IndexFlatIP index (inner product on normalized vectors = cosine similarity).
- Step 4: Accept a user query, validate it, embed it, and normalize it the same way.
- Step 5: Search the FAISS index for the nearest FAQ vector and read off its similarity score.
- Step 6: If the score is below a confidence threshold, return a fallback "could not find a match" message instead of a wrong answer.
- Step 7: Otherwise return the answer belonging to the matched FAQ.

4. Source Code

```
"""
Question 25 - College FAQ Retrieval System
Hands-on API Assignment - CO3

Uses Hugging Face sentence embeddings + a FAISS vector database to
build a semantic FAQ retrieval system for a college website. The
system returns the correct FAQ answer even when the student's
wording is different from the stored question.
"""

from sentence_transformers import SentenceTransformer
import faiss
import numpy as np

# -----
# 1. College FAQ knowledge base (11 FAQs)
# -----
faqs = [
    {"question": "What is the last date for admission?",
     "answer": "Admissions close on the 15th of August every academic year. Late admissions are accepted with a fine until 31st August."},
    {"question": "How can I pay my semester fees?",
     "answer": "Fees can be paid online through the college ERP portal using net banking, UPI, or debit/credit card, or offline at the accounts office."},
    {"question": "What is the minimum attendance required for exams?"}
```

```

"answer": "Students must maintain a minimum of 75% attendance to be eligible to write the semester examinations."},
{"question": "How do I apply for a scholarship?",
"answer": "Scholarship applications are submitted through the Student Welfare portal along with income and academic proof before the
deadline."},
{"question": "Is hostel accommodation available for first year students?",
"answer": "Yes, hostel accommodation is available for first year students on a first-come-first-served basis; applications open right after
admission."},
{"question": "What is the process for revaluation of answer scripts?",
"answer": "Students can apply for revaluation within 7 days of result announcement through the examination portal by paying the prescribed
fee."},
{"question": "What are the library working hours?",
"answer": "The central library is open from 8:00 AM to 8:00 PM on all working days and 9:00 AM to 1:00 PM on Saturdays."},
{"question": "How is the internal assessment calculated?",
"answer": "Internal assessment is calculated from two internal exams, assignments, and attendance, together contributing 25 marks out of
100."},
{"question": "Does the college provide transport facility?",
"answer": "Yes, the college operates buses on multiple routes across the city; the route list and fee structure are available at the transport
office."},
{"question": "What is the placement eligibility criteria?",
"answer": "Students need a minimum CGPA of 6.5 with no active backlogs to be eligible to sit for on-campus placement drives."},
{"question": "How can I get a bonafide certificate?",
"answer": "Bonafide certificates can be requested through the student portal and are usually issued within 3 working days from the department
office."}
]

```

```

questions = [item["question"] for item in faqs]

```

```

# -----
# 2. Load embedding model and encode FAQ questions
# -----
print("Loading embedding model...")
model = SentenceTransformer("all-MiniLM-L6-v2")
faq_embeddings = model.encode(questions)
faq_embeddings = np.array(faq_embeddings).astype("float32")

```

```

# Normalize vectors so that inner product = cosine similarity
faiss.normalize_L2(faq_embeddings)

```

```

# -----
# 3. Build the FAISS vector index
# -----
dimension = faq_embeddings.shape[1]
index = faiss.IndexFlatIP(dimension) # inner-product index
index.add(faq_embeddings)
print(f"FAISS index built with {index.ntotal} FAQ vectors of dimension {dimension}.")

```

```

def search_faq(user_query, top_k=1, threshold=0.35):
    """
    Embeds the user's query, searches the FAISS index, and returns the
    best matching FAQ answer. Returns a fallback message if no FAQ is
    a confident enough match, or if the query is empty.
    """
    if not user_query or not user_query.strip():
        return "Error: Please type a question.", None

    q_vec = model.encode([user_query]).astype("float32")
    faiss.normalize_L2(q_vec)

    scores, indices = index.search(q_vec, top_k)
    best_score = float(scores[0][0])
    best_idx = int(indices[0][0])

    if best_score < threshold:
        return ("Sorry, I could not find a matching FAQ for your question. "
                "Please contact the college helpdesk."), best_score

    matched_faq = faqs[best_idx]

```

```

return matched_faq["answer"], best_score

# -----
# 4. Demonstration with differently worded queries
# -----
if name == "main":
    user_queries = [
        "When does admission close this year?",
        "Can I stay in the hostel in my first year?",
        "Minimum CGPA needed for placements?",
        "What time does the library close?",
        "Do you sell movie tickets?", # out-of-scope query
        "" # empty input
    ]

    for q in user_queries:
        answer, score = search_faq(q)
        print(f"\nUser Query : {q} if q else '(empty)')")
        if score is not None:
            print(f'Match Score: {score:.4f}')
            print(f'System : {answer}')

```

5. Program Output (as executed)

```

Loading embedding model...
FAISS index built with 11 FAQ vectors of dimension 384.

User Query : When does admission close this year?
Match Score: 0.8127
System : Admissions close on the 15th of August every academic year. Late admissions are accepted with a fine until 31st August.

User Query : Can I stay in the hostel in my first year?
Match Score: 0.8659
System : Yes, hostel accommodation is available for first year students on a first-come-first-served basis; applications open right after admission.

User Query : Minimum CGPA needed for placements?
Match Score: 0.7934
System : Students need a minimum CGPA of 6.5 with no active backlogs to be eligible to sit for on-campus placement drives.

User Query : What time does the library close?
Match Score: 0.8412
System : The central library is open from 8:00 AM to 8:00 PM on all working days and 9:00 AM to 1:00 PM on Saturdays.

User Query : Do you sell movie tickets?
Match Score: 0.1146
System : Sorry, I could not find a matching FAQ for your question. Please contact the college helpdesk.

User Query : (empty)
System : Error: Please type a question.

```

6. Explanation of the Output

Every test query is worded differently from the stored FAQ question, yet the correct answer was retrieved in all four in-scope cases. For example, "When does admission close this year?" does not share most of its words with the stored question "What is the last date for admission?", but the FAISS search still returned the right answer with a high match score (0.8127) because the embeddings capture meaning rather than exact wording. The fifth query, "Do you sell movie tickets?", is unrelated to any FAQ in the knowledge base, and its match score (0.1146) correctly falls below the 0.35 threshold, so the system responds with a fallback message instead of forcing an incorrect answer onto an unrelated question. The final test confirms that an empty query is rejected before it even reaches the FAISS index.

RUBRICS FOR CALCULATION

Criteria	Weight age	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
API Integration Correctness	30%	Correct, efficient API integration; understands request/response flow	Correct integration with minor inefficiencies	Partial integration with functional errors	API integration non-functional / major errors
Problem-Solving Completeness	25%	Solves the given problem completely and robustly	Solves problem with minor issues	Partial solution to the problem	Does not solve the assigned problem
Code Quality	20%	Clean, well-structured, error-handled code	Reasonably clean code	Code works but poorly structured	Code is unreliable or unstructured
Input/Output Demo	15%	Clear demo showing inputs/outputs and edge cases	Demo covers the main use cases	Demo is incomplete	No functional demo presented
Code Documentation	10%	Well-commented code with clear README/setup steps	Adequate comments/instructions	Minimal documentation provided	No documentation provided

Marks Evaluation:

Question No.	API Integration Correctness(30%)	Problem-Solving Completeness (25%)	Code Quality(20%)	Input/Output Demo(15%)	Code Documentation (10%)	Total Marks (100)
Q1						
Q2						
Overall Total (100)						